

- 7 One key disadvantage of launching an open source application instead of a commercial application is that it's a lot harder to collect user feedback. Users can take your open source application and deploy it themselves, and you have no idea how the application is used.

It's important to remember that user feedback is user data. Leveraging user feedback requires the same cautions needed when leveraging any data. User privacy should be respected. Users have the right to know how their data is being used.

Extracting Conversational Feedback

Traditionally, feedback can be *explicit* or *implicit*. Explicit feedback is information users provide in response to explicit requests for feedback in the application, such as thumbs up/thumbs down, upvote/downvote, star rating, or a yes/no answer to the question “Did we solve your problem?” Explicit feedback is fairly standard across applications—there are only so many ways you can ask a person if they like something. Therefore, explicit feedback is better understood.

Implicit feedback is information inferred from user actions. For example, if someone buys a product recommended to them, it means it was a good recommendation. What can be considered implicit feedback depends on what actions a user can do within each application and is, therefore, highly application-dependent. Foundation models enable a new world of applications and, with them, many genres of implicit feedback.

The conversational interface that many AI applications use makes it easier for users to give feedback. Users can encourage good behaviors and correct errors the same way they would give feedback in daily dialogues. The language that a user uses to give directions to AI can convey feedback about both *the application's performance* and *the user's preference*.

As an example, imagine you're using an AI assistant to help you plan your trip to Australia. You ask the AI to find a hotel for three nights in Sydney. It responds with three recommendations as follows:

Here are three hotel suggestions in Sydney for a 3-night stay:

1. Historic Rocks Boutique Hotel (The Rocks)

- Price: ~\$400/night
- Neighborhood: Charming streets and close to iconic sights.

2. Stylish Surry Hills House Hotel (Surry Hills)

- Price: ~\$200/night
- Neighborhood: Trendy, with vibrant cafes and art galleries.

3. Chill Bondi Beachside Hotel (Bondi Beach)

- Price: ~\$300/night
- Neighborhood: Beachside, ideal for relaxation and city exploration.

How you respond to these three recommendations reveals your preference. For example, if you respond with “Yes book me the one close to galleries”, you show an interest in art. On the other hand, the response “Is there nothing under \$200?” reveals a price-conscious preference and suggests that the assistant doesn’t quite get you yet.

User feedback, extracted from conversations, can be used for evaluation, development, and personalization:

- Evaluation: derive metrics to monitor the application
- Development: train the future models or guide their development
- Personalization: personalize the application to each user

Implicit conversational feedback can be inferred from both the content of user messages and their patterns of communication. Because feedback is blended into daily conversations, it’s also challenging to extract. While intuition about conversational cues can help you devise an initial set of signals to look for, rigorous data analysis and user studies are necessary to understand.

While conversational feedback has enjoyed greater attention thanks to the popularity of conversational bots, it had been an active research area for several years before ChatGPT came out. The reinforcement learning community has been trying to get RL algorithms to learn from natural language feedback since the late 2010s, many of them with promising results; see [Fu et al. \(2019\)](#); [Goyal et al. \(2019\)](#); [Zhou and Small \(2020\)](#); and [Sumers et al. \(2020\)](#)). Natural language feedback is also of great interest for early conversational AI applications such as Amazon Alexa ([Ponnusamy et al., 2019](#); [Park et al., 2020](#)), Spotify’s voice control feature ([Xiao et al., 2021](#)), and Yahoo! Voice ([Hashimoto and Sassano, 2018](#)).

Natural language feedback

Feedback extracted from the content of messages is called natural language feedback. Here are a couple of natural language feedback signals that tell you how a conversation is going. It’s useful to track these signals in production to monitor your application’s performance.

Early termination. If a user terminates a response early, e.g., stopping a response generation halfway, exiting the app (for web and mobile apps), telling the model to stop (for voice assistants), or simply leaving the agent hanging (e.g., not responding to the agent with which option you want it to go ahead with), it’s likely that the conversation isn’t going well.

Error correction. If a user starts their follow-up with “No, ...” or “I meant, ...”, the model’s response is likely off the mark.

To correct errors, users might try to rephrase their requests. **Figure 10-12** shows an example of a user’s attempt to correct the model’s misunderstanding. Rephrase attempts can be detected using heuristics or ML models.

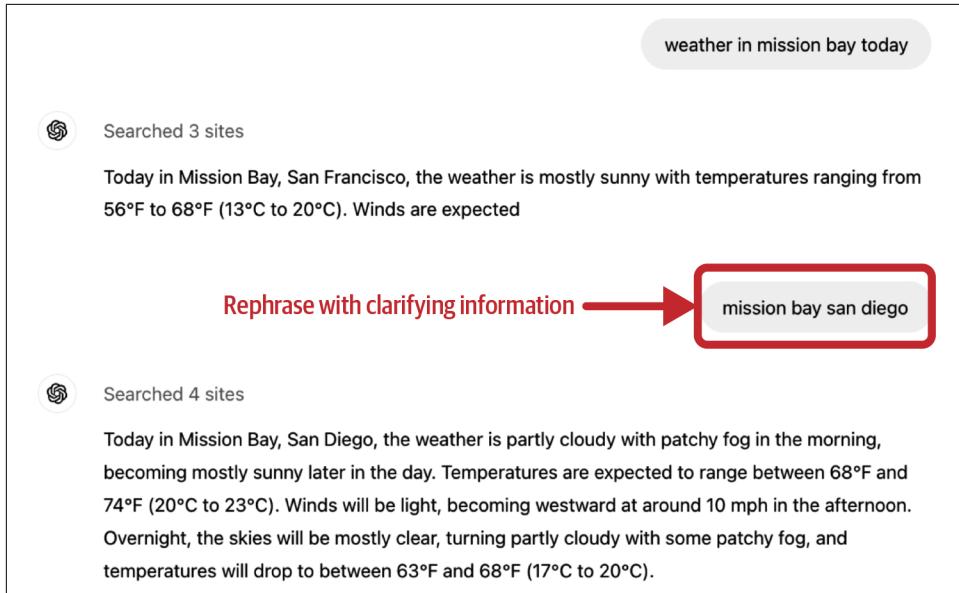


Figure 10-12. Because the user both terminates the generation early and rephrases the question, it can be inferred that the model misunderstood the intent of the original request.

Users can also point out specific things the model should’ve done differently. For example, if a user asks the model to summarize a story and the model confuses a character, this user can give feedback such as: “Bill is the suspect, not the victim.” The model should be able to take this feedback and revise the summary.

This kind of action-correcting feedback is especially common for agentic use cases where users might nudge the agent toward more optional actions. For example, if a user assigns the agent the task of doing market analysis about company XYZ, this user might give feedback such as “You should also check XYZ GitHub page” or “Check the CEO’s X profile”.

Sometimes, users might want the model to correct itself by asking for explicit confirmation, such as “Are you sure?”, “Check again”, or “Show me the sources”. This doesn’t necessarily mean that the model gives wrong answers. However, it might

mean that your model’s answers lack the details the user is looking for. It can also indicate general distrust in your model.

Some applications let users edit the model’s responses directly. For example, if a user asks the model to generate code, and the user corrects the generated code, it’s a very strong signal that the code that got edited isn’t quite right.

User edits also serve as a valuable source of preference data. Recall that preference data, typically in the format of (query, winning response, losing response), can be used to align a model to human preference. Each user edit makes up a preference example, with the original generated response being the losing response and the edited response being the winning response.

Complaints. Often, users just complain about your application’s outputs without trying to correct them. For example, they might complain that an answer is wrong, irrelevant, toxic, lengthy, lacking detail, or just bad. **Table 10-1** shows eight groups of natural language feedback resulting from automatic clustering the FITS (Feedback for Interactive Talk & Search) dataset (Xu et al., 2022).

Table 10-1. Feedback types derived from automatic clustering the FITS dataset (Xu et al., 2022). Results from Yuan et al. (2023).

Group	Feedback type	Num.	%
1	Clarify their demand again.	3702	26.54%
2	Complain that the bot (1) does not answer the question or (2) gives irrelevant information or (3) asks the user to find out the answer on their own.	2260	16.20%
3	Point out specific search results that can answer the question.	2255	16.17%
4	Suggest that the bot should use the search results.	2130	15.27%
5	State that the answer is (1) factually incorrect, or (2) not grounded in the search results.	1572	11.27%
6	Point out that the bot’s answer is not specific/accurate/complete/detailed.	1309	9.39%
7	Point out that the bot is not confident in its answers and always begins its responses with “I am not sure” or “I don’t know”.	582	4.17%
8	Complain about repetition/rudeness in bot responses.	137	0.99%

Understanding how the bot fails the user is crucial in making it better. For example, if you know that the user doesn’t like verbose answers, you can change the bot’s prompt to make it more concise. If the user is unhappy because the answer lacks details, you can prompt the bot to be more specific.

Sentiment. Complaints can also be general expressions of negative sentiments (frustration, disappointment, ridicule, etc.) without explaining the reason why, such as “Uggh”. This might sound dystopian, but analysis of a user’s sentiments throughout conversations with a bot might give you insights into how the bot is doing. Some call centers track users’ voices throughout the calls. If a user gets increasingly loud,

something is wrong. Conversely, if someone starts a conversation angry but ends happily, the conversation might have resolved their issue.

Natural language feedback can also be inferred from the model's responses. One important signal is the model's *refusal rate*. If a model says things like "Sorry, I don't know that one" or "As a language model, I can't do ...", the user is probably unhappy.

Other conversational feedback

Other types of conversational feedback can be derived from user actions instead of messages.

Regeneration. Many applications let users generate another response, sometimes with a different model. If a user chooses regeneration, it might be because they're not satisfied with the first response. However, it might also be that the first response is adequate, but the user wants options to compare. This is especially common with creative requests like image or story generation.

Regeneration signals might also be stronger for applications with usage-based billing than those with subscriptions. With usage-based billing, users are less likely to regenerate and spend extra money out of idle curiosity.

Personally, I often choose regeneration for complex requests to ensure the model's responses are consistent. If two responses give contradicting answers, I can't trust either.

After regeneration, some applications might explicitly ask to compare the new response with the previous one, as shown in [Figure 10-13](#). This better or worse data, again, can be used for preference finetuning.

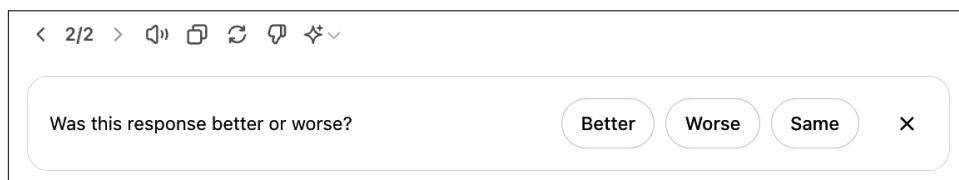


Figure 10-13. ChatGPT asks for comparative feedback when a user regenerates another response.

Conversation organization. The actions a user takes to organize their conversations—such as delete, rename, share, and bookmark—can also be signals. Deleting a conversation is a pretty strong signal that the conversation is bad, unless it's an embarrassing conversation and the user wants to remove its trace. Renaming a conversation suggests that the conversation is good, but the auto-generated title is bad.

Conversation length. Another commonly tracked signal is *the number of turns per conversation*. Whether this is a positive or negative signal depends on the application. For AI companions, a long conversation might indicate that the user enjoys the conversation. However, for chatbots geared toward productivity like customer support, a long conversation might indicate that the bot is inefficient in helping users resolve their issues.

Dialogue diversity. Conversation length can also be interpreted together with *dialogue diversity*, which can be measured by the distinct token or topic count. For example, if the conversation is long but the bot keeps repeating a few lines, the user might be stuck in a loop.

Explicit feedback is easier to interpret, but it demands extra effort from users. Since many users may not be willing to put in this additional work, explicit feedback can be sparse, especially in applications with smaller user bases. Explicit feedback also suffers from response biases. For example, unhappy users might be more likely to complain, causing the feedback to appear more negative than it is.

Implicit feedback is more abundant—what can be considered implicit feedback is limited only by your imagination—but it’s noisier. Interpreting implicit signals can be challenging. For example, sharing a conversation can either be a negative or a positive signal. For example, one friend of mine mostly shares conversations when the model has made some glaring mistakes, and another friend mostly shares useful conversations with their coworkers. *It’s important to study your users to understand why they do each action.*

Adding more signals can help clarify the intent. For example, if the user rephrases their question after sharing a link, it might indicate that the conversation didn’t meet their expectations. Extracting, interpreting, and leveraging implicit responses from conversations is a small but growing area of research.⁸

Feedback Design

If you were unsure of what feedback to collect, I hope that the last section gave you some ideas.

This section discusses when and how to collect this valuable feedback.

⁸ Not only can you collect feedback about AI applications, you can use AI to analyze feedback, too.

When to collect feedback

Feedback can and should be collected throughout the user journey. Users should have the option to give feedback, especially to report errors, whenever this need arises. The feedback collection option, however, should be nonintrusive. It shouldn't interfere with the user workflow. Here are a few places where user feedback might be particularly valuable.

In the beginning. When a user has just signed up, user feedback can help calibrate the application for the user. For example, a face ID app first must scan your face to work. A voice assistant might ask you to read a sentence out loud to recognize your voice for wake words (words that activate a voice assistant, like “Hey Google”). A language learning app might ask you a few questions to gauge your skill level. For some applications, such as face ID, calibration is necessary. For other applications, however, initial feedback should be optional, as it creates friction for users to try out your product. If a user doesn't specify their preference, you can fall back to a neutral option and calibrate over time.

When something bad happens. When the model hallucinates a response, blocks a legitimate request, generates a compromising image, or takes too long to respond, users should be able to notify you of these failures. You can give users the option to downvote a response, regenerate with the same model, or change to another model. Users might just give conversational feedback like “You're wrong”, “Too cliché”, or “I want something shorter”.

Ideally, when your product makes mistakes, users should still be able to accomplish their tasks. For example, if the model wrongly categorizes a product, users can edit the category. Let users collaborate with the AI. If that doesn't work, let them collaborate with humans. Many customer support bots offer to transfer users to human agents if the conversation drags on or if users seem frustrated.

An example of human–AI collaboration is the *inpainting* functionality for image generation.⁹ If a generated image isn't exactly what the user needs, they can select a region of the image and describe with a prompt how to make it better. **Figure 10-14** shows an example of inpainting with **DALL-E** (OpenAI, 2021). This feature allows users to get better results while giving developers high-quality feedback.

⁹ I wish there were inpainting for text-to-speech. I find text-to-speech works well 95% of the time, but the other 5% can be frustrating. AI might mispronounce a name or fail to pause during dialogues. I wish there were apps that let me edit just the mistakes instead of having to regenerate the whole audio.

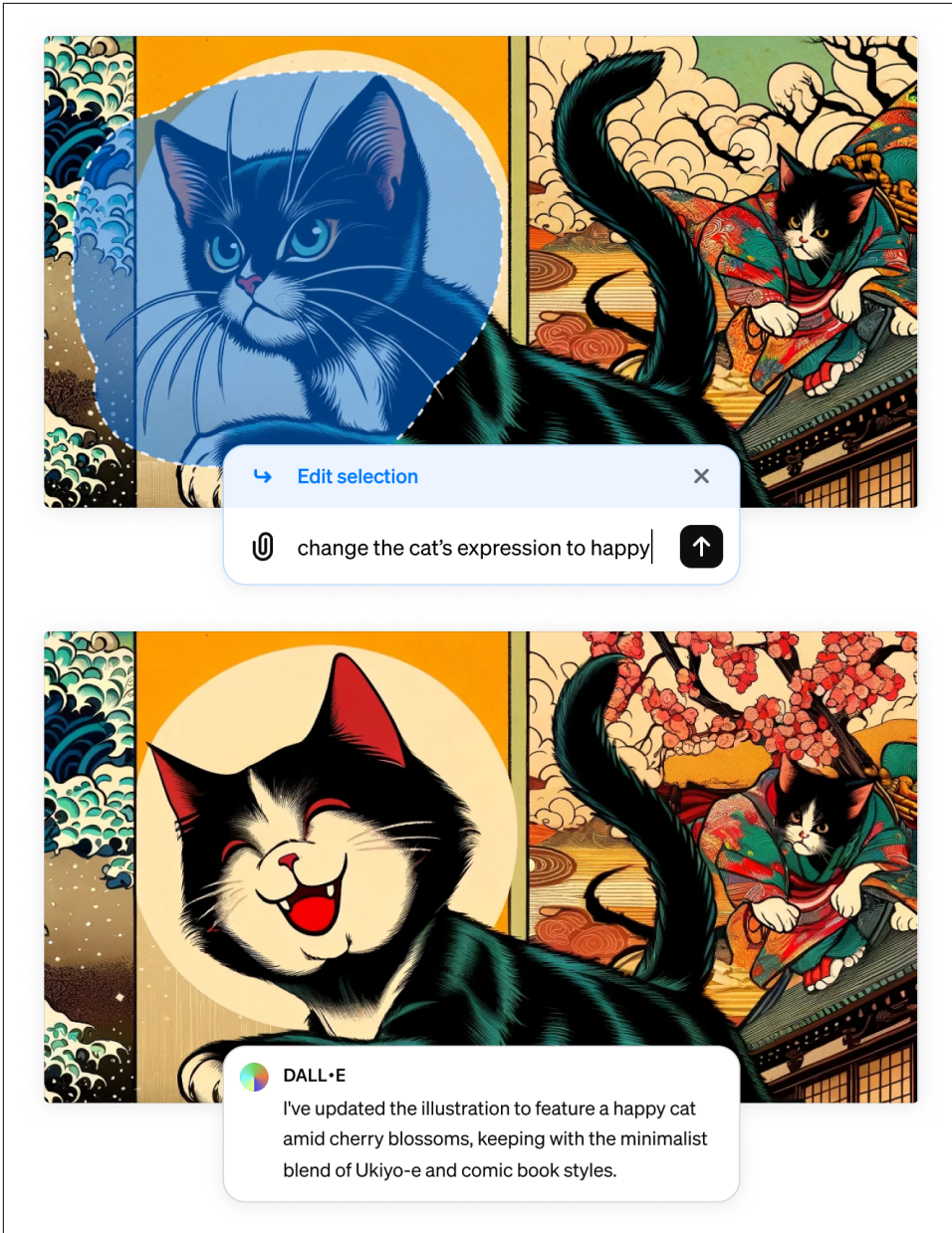


Figure 10-14. An example of how inpainting works in DALL-E. Image by *OpenAI*.

When the model has low confidence. When a model is uncertain about an action, you can ask the user for feedback to increase its confidence. For example, given a request to summarize a paper, if the model is uncertain whether the user would prefer a short, high-level summary or a detailed section-by-section summary, the model can output both summaries side by side, assuming that generating two summaries doesn't increase the latency for the user. The user can choose which one they prefer. Comparative signals like this can be used for preference finetuning. An example of comparative evaluation in production is shown in [Figure 10-15](#).

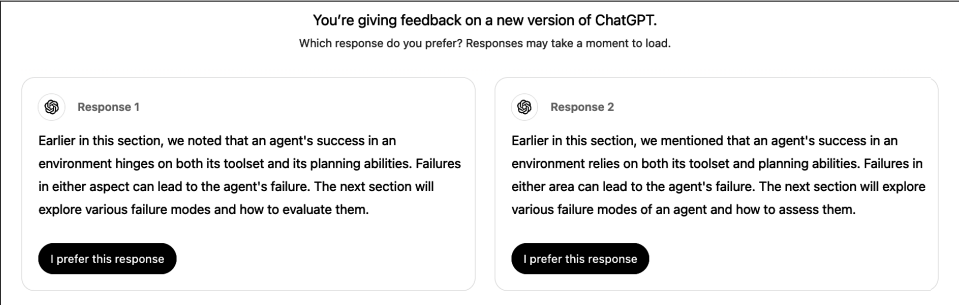


Figure 10-15. Side-by-side comparison of two ChatGPT responses.

Showing two full responses for the user to choose means asking that user for explicit feedback. Users might not have time to read two full responses or care enough to give thoughtful feedback. This can result in noisy votes. Some applications, like Google Gemini, show only the beginning of each response, as shown in [Figure 10-16](#). Users can click to expand the response they want to read. It's unclear, however, whether showing full or partial responses side by side gives more reliable feedback.¹⁰

¹⁰ When I ask this question at events I speak at, the responses are conflicted. Some people think showing full responses gives more reliable feedback because it gives users more information to make a decision. At the same time, some people think that once users have read full responses, there's no incentive for them to click on the better one.

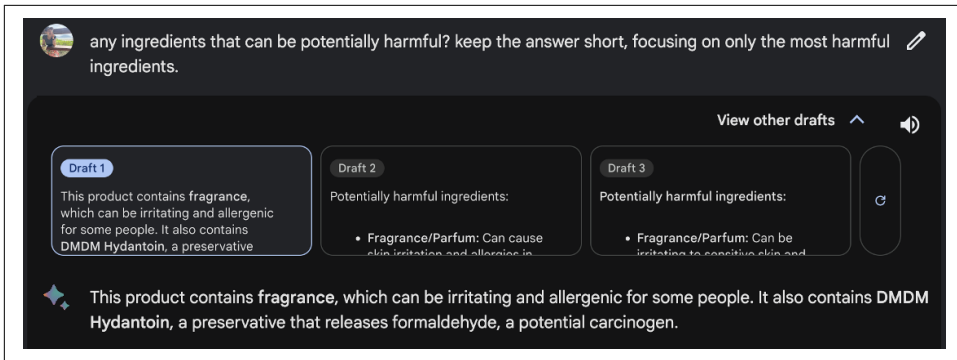


Figure 10-16. Google Gemini shows partial responses side by side for comparative feedback. Users have to click on the response they want to read more about, which gives feedback about which response they find more promising.

Another example is a photo organization application that automatically tags your photos, so that it can respond to queries like “Show me all the photos of X”. When unsure if two people are the same, it can ask you for feedback, as Google Photos does in Figure 10-17.

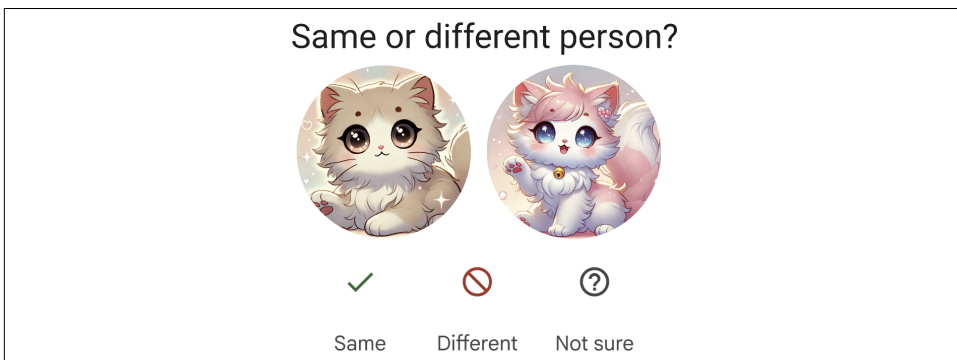


Figure 10-17. Google Photos asks for user feedback when unsure. The two cat images were generated by ChatGPT.

You might wonder: how about feedback when something good happens? Actions that users can take to express their satisfaction include thumbs up, favoriting, or sharing. However, Apple’s **human interface guideline** warns against asking for both positive and negative feedback. Your application should produce good results by default. Asking for feedback on good results might give users the impression that good results are exceptions. Ultimately, if users are happy, they continue using your application.

However, many people I've talked to believe users should have the option to give feedback when they encounter something amazing. A product manager for a popular AI-powered product mentioned that their team needs positive feedback because it reveals the features users love enough to give enthusiastic feedback about. This allows the team to concentrate on refining a small set of high-impact features rather than spreading resources across many with minimal added value.

Some avoid asking for positive feedback out of concern it may clutter the interface or annoy users. However, this risk can be managed by limiting the frequency of feedback requests. For example, if you have a large user base, showing the request to only 1% of users at a time could help gather sufficient feedback without disrupting the experience for most users. Keep in mind that the smaller the percentage of users asked, the greater the risk of feedback biases. Still, with a large enough pool, the feedback can provide meaningful product insights.

How to collect feedback

Feedback should seamlessly integrate into the user's workflow. It should be easy for users to provide feedback without extra work. Feedback collection shouldn't disrupt user experience and should be easy to ignore. There should be incentives for users to give good feedback.

One example often cited as good feedback design is from the image generator app Midjourney. For each prompt, Midjourney generates a set of (four) images and gives the user the following options, as shown in [Figure 10-18](#):

1. Generate an unscaled version of any of these images.
2. Generate variations for any of these images.
3. Regenerate.

All these options give Midjourney different signals. Options 1 and 2 tell Midjourney which of the four photos is considered by the user to be the most promising. Option 1 gives the strongest positive signal about the chosen photo. Option 2 gives a weaker positive signal. Option 3 signals that none of the photos is good enough. However, users might choose to regenerate even if the existing photos are good just to see what else is possible.

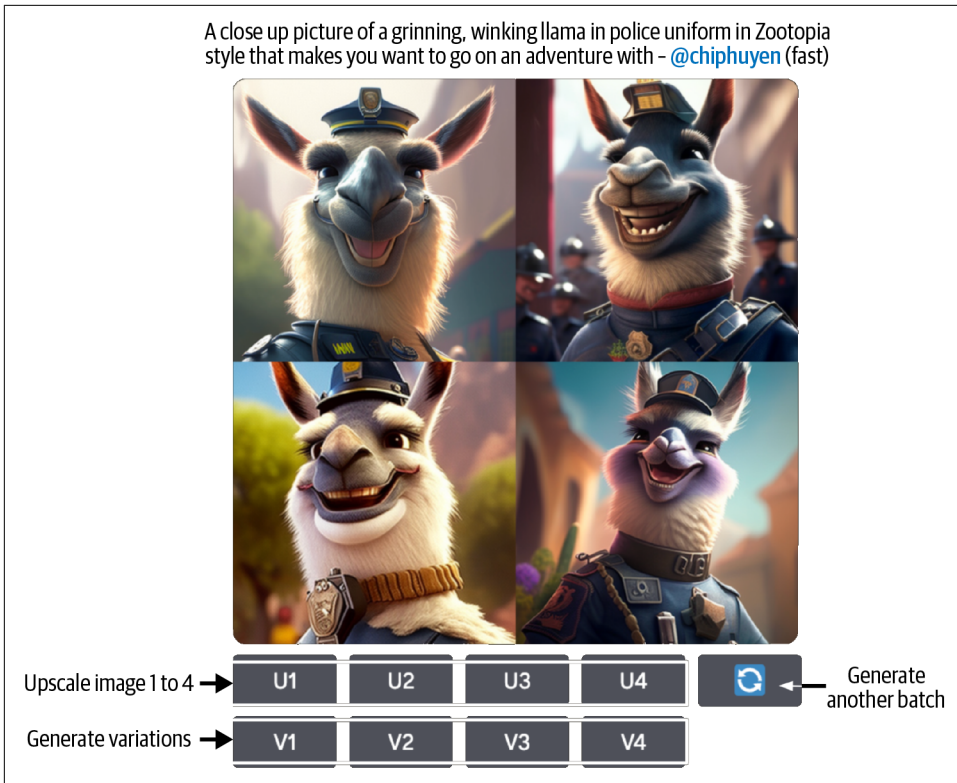


Figure 10-18. Midjourney's workflow allows the app to collect implicit feedback.

Code assistants like GitHub Copilot might show their drafts in lighter colors than the final texts, as shown in Figure 10-19. Users can use the Tab key to accept a suggestion or simply continue typing to ignore the suggestion, both providing feedback.

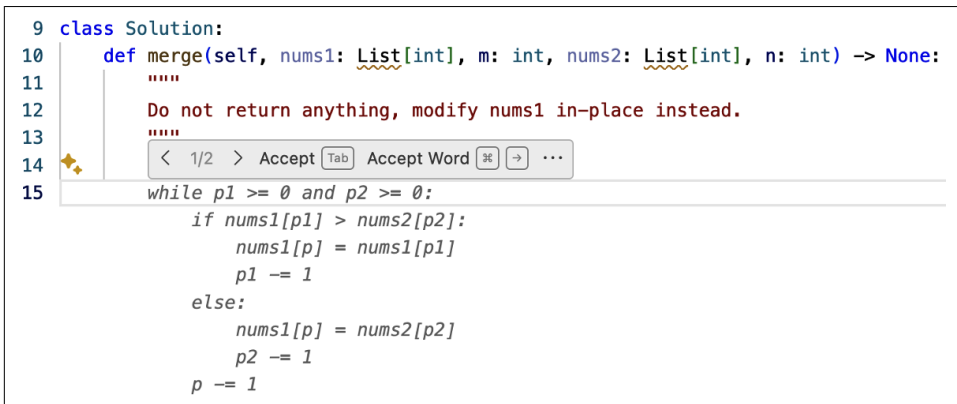


Figure 10-19. GitHub Copilot makes it easy to both suggest and reject a suggestion.

One of the biggest challenges of standalone AI applications like ChatGPT and Claude is that they aren't integrated into the user's daily workflow, making it hard to collect high-quality feedback the way integrated products like GitHub Copilot can. For example, if Gmail suggests an email draft, Gmail can track how this draft is used or edited. However, if you use ChatGPT to write an email, ChatGPT doesn't know whether the generated email is actually sent.

The feedback alone might be helpful for product analytics. For example, seeing just the thumbs up/thumbs down information is useful for calculating how often people are happy or unhappy with your product. For deeper analysis, though, you would need context around the feedback, such as the previous 5 to 10 dialogue turns. This context can help you figure out what went wrong. However, getting this context might not be possible without explicit user consent, especially if the context might contain personally identifiable information.

For this reason, some products include terms in their service agreements that allow them to access user data for analytics and product improvement. For applications without such terms, user feedback might be tied to a user data donation flow, where users are asked to donate (e.g., share) their recent interaction data along with their feedback. For example, when submitting feedback, you might be asked to check a box to share your recent data as context for this feedback.

Explaining to users how their feedback is used can motivate them to give more and better feedback. Do you use a user's feedback to personalize the product to this user, to collect statistics about general usage, or to train a new model? If users are concerned about privacy, reassure them that their data won't be used to train models or won't leave their device (only if these are true).

Don't ask users to do the impossible. For example, if you collect comparative signals from users, don't ask them to choose between two options they don't understand. For example, I was once stumped when ChatGPT asked me to choose between two possible answers to a statistical question, as shown in [Figure 10-20](#). I wish there was an option for me to say, "I don't know".

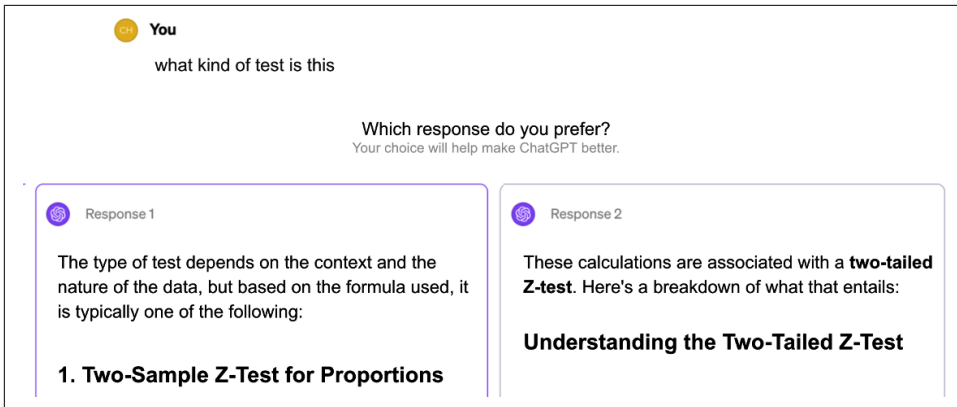


Figure 10-20. An example of ChatGPT asking a user to select the response the user prefers. However, for mathematical questions like this, the right answer shouldn't be a matter of preference.

Add icons and tooltips to an option if they help people understand it. Avoid a design that can confuse users. Ambiguous instructions can lead to noisy feedback. I once hosted a GPU optimization workshop, using Luma to collect feedback. When I was reading the negative feedback, I was confused. Even though the responses were positive, the star ratings were 1/5. When I dug deeper, I realized that Luma used emojis to represent numbers in their feedback collection form, but the angry emoji, corresponding to a one-star rating, was put where the five-star rating should be, as shown in Figure 10-21.

Be mindful of whether you want users' feedback to be private or public. For example, if a user likes something, do you want this information shown to other users? In its early days, Midjourney's feedback—someone choosing to upscale an image, generate variations, or regenerate another batch of images—was public.

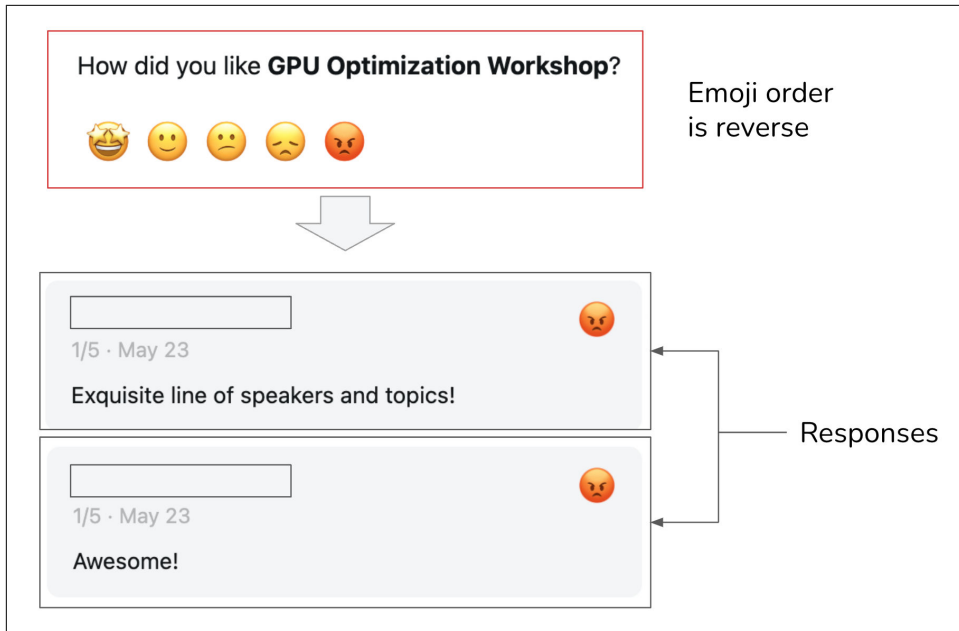


Figure 10-21. Because Luma put the angry emoji, corresponding to a one-star rating, where a five-star rating should've been, some users mistakenly picked it for positive reviews.

The visibility of a signal can profoundly impact user behavior, user experience, and the quality of the feedback. Users tend to be more candid in private—there's a lower chance of their activities being judged¹¹—which can result in higher-quality signals. In 2024, X (formerly Twitter) made “likes” **private**. Elon Musk, the owner of X, claimed a significant **uptick in the number of likes** after this change.

However, private signals can reduce discoverability and explainability. For example, hiding likes prevents users from finding tweets their connections have liked. If X recommends tweets based on the likes of the people you follow, hiding likes could result in users' confusion about why certain tweets appear in their feeds.

¹¹ See “Ted Cruz Blames Staffer for ‘Liking’ Porn Tweet” (Nelson and Everett, *POLITICO*, September 2017) and “Kentucky Senator Whose Twitter Account ‘Liked’ Obscene Tweets Says He Was Hacked” (Liam Niemeyer, WKU Public Radio, March 2023).

Feedback Limitations

There's no doubt of the value of user feedback to an application developer. However, feedback isn't a free lunch. It comes with its own limitations.

Biases

Like any other data, user feedback has biases. It's important to understand these biases and design your feedback system around them. Each application has its own biases. Here are a few examples of feedback biases to give you an idea of what to look out for:

Leniency bias

Leniency bias is the tendency for people to rate items more positively than warranted, often to avoid conflict because they feel compelled to be nice or because it's the easiest option. Imagine you're in a hurry, and an app asks you to rate a transaction. You aren't happy with the transaction, but you know that if you rate it negatively, you'll be asked to provide reasons, so you just choose positive to be done with it. This is also why you shouldn't make people do extra work for your feedback.

On a five-star rating scale, four and five stars are typically meant to indicate a good experience. However, in many cases, users may feel pressured to give five-star ratings, reserving four stars for when something goes wrong. According to [Uber](#), in 2015, the average driver's rating was 4.8, with scores below 4.6 putting drivers at risk of being deactivated.

This bias isn't necessarily a dealbreaker. Uber's goal is to differentiate good drivers from bad drivers. Even with this bias, their rating system seems to help them achieve this goal. It's essential to look at the distribution of your user ratings to detect this bias.

If you want more granular feedback, removing the strong negative connotation associated with low ratings can help people break out of this bias. For example, instead of showing users numbers one to five, show users options such as the following:

- "Great ride. Great driver."
- "Pretty good."
- "Nothing to complain about but nothing stellar either."
- "Could've been better."
- "Don't match me with this driver again."¹²

¹² The options suggested here are only to show how options can be rewritten. They haven't been validated.

Randomness

Users often provide random feedback, not out of malice, but because they lack motivation to give more thoughtful input. For example, when two long responses are shown side by side for comparative evaluation, users might not want to read both of them and just click on one at random. In the case of Midjourney, users might also randomly choose one image to generate variations.

Position bias

The position in which an option is presented to users influences how this option is perceived. Users are generally more likely to click on the first suggestion than the second. If a user clicks on the first suggestion, this doesn't necessarily mean that it's a good suggestion.

When designing your feedback system, this bias can be mitigated by randomly varying the positions of your suggestions or by building a model to compute a suggestion's true success rate based on its position.

Preference bias

Many other biases can affect a person's feedback, some of which have been discussed in this book. For example, people might prefer the longer response in a side-by-side comparison, even if the longer response is less accurate—length is easier to notice than inaccuracies. Another bias is *recency bias*, where people tend to favor the answer they see last when comparing two answers.

It's important to inspect your user feedback to uncover its biases. Understanding these biases will help you interpret the feedback correctly, avoiding misleading product decisions.

Degenerate feedback loop

Keep in mind that user feedback is incomplete. You only get feedback on what you show users.

In a system where user feedback is used to modify a model's behavior, *degenerate feedback loops* can arise. A degenerate feedback loop can happen when the predictions themselves influence the feedback, which, in turn, influences the next iteration of the model, amplifying initial biases.

Imagine you're building a system to recommend videos. The videos that rank higher show up first, so they get more clicks, reinforcing the system's belief that they're the best picks. Initially, the difference between the two videos, A and B, might be minor, but because A was ranked slightly higher, it got more clicks, and the system kept boosting it. Over time, A's ranking soared, leaving B behind. This feedback loop is why popular videos stay popular, making it tough for new ones to break through. This issue is known as "exposure bias," "popularity bias," or "filter bubbles," and it's a well-studied problem.